

Nombre:						Turno:			
Calificación:			+				+		=
Importante - La duración del examen es de 2 horas y 40 minutos, a partir de la entrega del ejercicio. - Está terminantemente prohibido sacar del aula el examen o una copia de este. - Sólo se corregirán las preguntas escritas con bolígrafo. - Excepto las preguntas tipo test, desarrolle cada pregunta en hoja separada. - Comentar adecuadamente el código o pseudo-código, así como las variables utilizadas.									

1. **Preguntas tipo test** (Cada respuesta correcta tiene una puntuación de **0'2**. Por **cada respuesta incorrecta se descontará 0'1**. Solo serán válidas las respuestas marcadas dentro de las casillas)
 - 1.1. Cuál es el resultado del siguiente código


```

class A {
public:
    A() : { std::cout << "A" << std::endl; }
    ~A() : { std::cout << "~A" << std::endl; }
};
class B : public A {
    B() : { std::cout << "B" << std::endl; }
    ~B() : { std::cout << "~B" << std::endl; }
};

int main() {
    A *a = new B;
    delete a;
    return 0;
}
          
```

☐ AB~A~B
 ☐ BA~B~A
 ☐ ~B~ABA
 ☐ BA~A~B
 - 1.2. Cuál es la característica del polimorfismo en C++

☐ Capacidad de una función para cambiar su firma
 ☐ Capacidad de una clase para heredar de múltiples clases
 ☐ Capacidad de una función para tomar varios valores sobrecargandose
 ☐ Capacidad de un puntero o referencia de clase base para invocar funciones de una clase determinada
 - 1.3. Cual de las siguientes define correctamente una template specialization completa en c++

☐ template<typename T> class MyClass {}
 ☐ template<> class MyClass<int> {}
 ☐ template<class T> void MyFunction {}
 ☐ template<typename T> class MyClass<T> {}
 - 1.4. Cual es el propósito de Noexcept

☐ Declarar funciones de plantilla
 ☐ Indicar que una función no lanza una excepción
 ☐ Hacer una función inline

- ☐ Definir una función virtual
- 1.5. MergeSort las 2 partes en las que se divide
 - ☐ Son siempre de igual tamaño
 - ☐ La diferencia entre tamaño es siempre 1
 - ☐ La diferencia entre tamaño es 0 o 1
 - ☐ Ninguna de las anteriores es correcta
- 1.6. Cual de los siguientes algoritmos de ordenación es logarítmica
 - ☐ Selection
 - ☐ Insertion
 - ☐ Mezcla
 - ☐ Intercambio
- 1.7. La búsqueda binaria requiere que
 - ☐ Los elementos estén en base 2
 - ☐ Los elementos estén ordenados
 - ☐ El tamaño de la secuencia o array sea par
 - ☐ El tamaño de la secuencia o array sea potencia de 2
- 1.8. Si durante la ordenación de selección de la secuencia [11, 12, 29, 32, 23, 15, 30] ¿Cuál es el resultado en la siguiente selección?
 - ☐ 11, 12, 15, 29, 32, 23, 30
 - ☐ 11, 12, 15, 23, 29, 32, 30
 - ☐ 11, 12, 29, 32, 23, 15, 30
 - ☐ 11, 12, 29, 32, 15, 23, 30
- 1.9. Insertar o eliminar un elemento de un árbol binario equilibrado
 - ☐ Siempre hay que reestructurar el árbol usando rotaciones
 - ☐ Si el árbol se desbalancea hay que reestructurar usando rotaciones
 - ☐ Si el árbol se desequilibrado hay que reestructurar usando rotaciones
 - ☐ No se usa radiación para reestructurar el árbol
- 1.10. Dado el recorrido preorden [1, 5, 10, 15, 20, 25]
 - ☐ 1 es la raíz
 - ☐ 15 es la raíz
 - ☐ 20 es la raíz
 - ☐ 25 es la raíz
- 2. **Preguntas cortas** (Cada pregunta es de 1 punto, desarrolle cada una en una hoja separada)
- 2.1. Describe brevemente RadixSort, y ordena [218, 411, 371, 457, 108, 187, 351, 158], mostrar la menor, en cada iteración los números de las cubetas o piletas antes de extraer los números y el estado de la secuencia después de extraer, hasta que la secuencia quede ordenada

- 2.2. La disposición de elementos de la tabla hash con el DNI como clave, la función de dispersión es la suma de los dos últimos dígitos y la de exploración de dispersión doble basada en la suma de los dos primeros de la clave:

0	1	2	3	4
LUIS		ANDRÉS	ANA	JESUS
PEDRO			AINARA	

Determina la disposición de la tabla al introducir, las fichas siguientes en el orden en el que aparecen:

PAULA: 45024658
 JON: 56894366
 LAURA: 54274812
 PINO: 47758764
 SARA: 41344205
 LUCAS: 43856662

- 2.3. Indica y justifica cuál es el resultado que aparece por pantalla

```
class A {
private:
    int a_;
public:
    A(int x) : a_(x) {}
    virtual int f(int x) { return g(x * x) }
    virtual int g(int x) const { return x * a_; }
};
class B : public A {
private:
    int b_;
public:
    B(int x = 2) : A(x * x), b_(x) {}
    int f(int x) const { return g(x + b_); }
};
class C : public B {
public:
    int f(int x) { return A::f(x + x); }
    int g(int x) const { return x; }
};
int main() {
    B *a = new B;
    std::cout << a->f(4) << std::endl;
    A *b = new B(5);
    std::cout << b->f(3) << std::endl;
    A *c = new C;
    std::cout << c->f(2) << std::endl;
    return 0;
}
```

- 2.4. Un árbol AVL inserta elementos siguiendo el orden que aquí se indica: 80, 50, 70, 30, 20, 10, 60. A continuación se eliminan en este orden: 30, 20, 50. Mostrar la traza de cada operación de inserción y eliminación.

En operaciones se produce desbalance, indica el nodo desbalanceado, el tipo de rotación (LL, RR, LR, RL), mostrar el árbol antes y después.

3. **Preguntas de desarrollo** (Cada pregunta es de 2 puntos, desarrolle cada una en una hoja separada)

- 3.1. Una función de C++ ordena la primera mitad de la secuencia o vector por selección de mayor a menor, y la segunda mitad por inserción de menor a mayor, pero inserta los elementos por la derecha, finalmente mezcla las mitades quedando ordenado de menor a mayor.

- 3.2. El árbol binario, AB, es un caso particular de árbol N-ario que se define como árbol cuando cada nodo puede tener como máximo N hijos. La familia N-arios se especifica mediante la clase abstracta, NTree<class Key, int N> donde el valor N indica el número máximo de hijos del node y key es el tipo de dato almacenado, Por tanto AB<Key> continente datos Key es el árbol N-ario para N=2, NTree(Key, 2).

Como se quiere escribir un programa que trabajará simultáneamente con diferentes hijos de árboles N-arios, se especifica la clase base abstracta Tree, de la cual se deriva, por herencia pública la plantilla de clases NTree<class Key, int N>. En esta clase abstracta sólo se especifican los métodos nulos para operaciones con cualquier hijo del árbol, buscar, insertar, recorrer por niveles, y destructor.

1. Escribe la especificación de la clase abstracta Tree
2. escribe la declaración (sólo los atributos y prototipo de los métodos) para la plantilla de clase abstracta NTree<class Key, int N>, donde las operaciones de busca e insertar son nulas, pero se implementa recorrido por niveles.

La plantilla Nodo<class Key, int N> es la generalización de clase nodo binario NodoB<Key> contiene:

- Atributo protegido nKey es el número de datos tipo key almacenado en el Nodo, se verifica que $0 \leq nKey \leq N$
- Atributo protegido Keys es el array que contiene nkeys del tipo key.
- Atributo protegido Links es el array de N punteros a nKeys contiene enlaces a los N nodos.
- Constructor que inicializa el nodo con dato Key y los N punteros a nulo
- Método público get(int i) tal que $0 \leq i < Keys$, retorna el i-ésimo elemento del array Key, si i no contiene datos genera una excepción del tipo NodeException
- Método público link(int i) tal que $0 \leq i < N$, retorna el i-ésimo enlace del array Links, si en la posición i no hay enlace genera una excepción del tipo NodeException

3. Se pide escribir la especificación de plantilla Node<class Key, int N>
4. Escribir la declaración (solo sentence class) árbol binario de búsqueda ABB<Key> como caso particular del árbol N-ario con N=2 con claves tipo Key
5. Escribe el programa principal mediante punteros a la clase base Tree para que acceda a un objeto ABB<Key> introducido mediante memoria dinámica. El programa inserta los valores 10, 20, luego busca el valor clave 30, y finalmente realiza una traza por nivelar. El programa captura y maneja cualquier excepción generada durante la ejecución de las operaciones